Pendahuluan

Perkembangan teknologi informasi saat ini menghasilkan data dalam jumlah yang sangat besar dengan karakteristik yang kompleks dan beragam. Kondisi ini membuat proses analisis menjadi semakin menantang, terutama ketika data tersebut tidak memiliki label. Oleh karena itu, diperlukan metode yang mampu mengolah data sekaligus menemukan pola secara mandiri tanpa bergantung pada informasi sebelumnya.

Salah satu metode yang dapat digunakan adalah ***clustering.*** Metode ini memungkinkan data dikelompokkan secara otomatis berdasarkan tingkat kemiripan karakteristiknya. Dengan kemampuan tersebut, *cluster*ing menjadi sangat bermanfaat dan banyak diterapkan di berbagai bidang, seperti kesehatan, bisnis, hingga teknologi, khususnya dalam proses segmentasi untuk memahami pola tersembunyi dalam data.

Secara umum, cara kerja *cluster*ing dimulai dengan menentukan jumlah *cluster*, kemudian algoritma menghitung jarak atau tingkat kesamaan antar data, lalu menempatkan data ke dalam kelompok yang paling sesuai. Pada beberapa metode, proses ini dilakukan secara berulang hingga posisi *cluster* menjadi stabil. Hasil akhirnya adalah kelompok-kelompok data yang memiliki kemiripan tinggi dalam satu *cluster* dan perbedaan yang jelas dengan *cluster* lainnya, sehingga memudahkan dalam mengidentifikasi pola tanpa memerlukan label data.

Dalam proses tersebut, perhitungan jarak menjadi aspek yang sangat penting karena menjadi dasar dalam pembentukan kelompok dari beberapa variabel yang digunakan. Hal ini menjadikan **standarisasi data** sebagai tahapan yang tidak dapat diabaikan, terutama untuk mengatasi perbedaan skala antar variabel. Tanpa standarisasi, variabel dengan rentang nilai yang lebih besar cenderung mendominasi perhitungan jarak dan berpotensi menghasilkan pengelompokan yang kurang akurat. Oleh karena itu, standarisasi diperlukan agar setiap variabel memiliki kontribusi yang seimbang, sehingga hasil *cluster*ing menjadi lebih representatif dan reliabel. Selain itu, terdapat dua prinsip utama yang menjadi dasar dalam *cluster*ing, yaitu meminimalkan variasi dalam satu *cluster* (intra-*cluster*) dan memaksimalkan perbedaan antar *cluster* (inter-*cluster*). Dengan memenuhi kedua prinsip ini, kualitas pengelompokan data akan menjadi lebih baik dan lebih mudah diinterpretasikan.

Berdasarkan pendekatannya, *cluster*ing memiliki beberapa jenis yang umum digunakan, yaitu:

1. Density-based *cluster*ing, yang mengelompokkan data berdasarkan kepadatan sehingga mampu mendeteksi *cluster* dengan bentuk yang tidak beraturan serta dapat mengidentifikasi noise.
2. *Hierarchical Clustering*, yaitu metode yang membentuk struktur bertingkat berupa dendrogram, baik secara aglomeratif (menggabungkan data dari bawah) maupun divisif (memecah dari atas).
3. Partitioning/ *non-hierarchical cluster*ing, yang membagi data ke dalam sejumlah *cluster* yang telah ditentukan sebelumnya.

Pemilihan jenis *cluster*ing ini umumnya disesuaikan dengan karakteristik data dan tujuan analisis yang ingin dicapai. Pada materi ini hanya membahas dua jenis saja, yaitu hierarchical dan partitioning *cluster*ing.

Hierarchical Clustering

A. Pendahuluan

Hierarchical clustering merupakan salah satu metode dalam unsupervised learning yang digunakan untuk mengelompokkan data berdasarkan tingkat kemiripan (*similarity*) atau perbedaan (*dissimilarity*) antar objek. Metode ini membentuk struktur bertingkat yang divisualisasikan dalam bentuk dendrogram. Dendrogram tersebut menggambarkan proses penggabungan atau pemisahan data yang terjadi secara bertahap, sehingga hubungan antar objek dapat terlihat dengan lebih jelas.

Karakteristik *Hierarchical Clustering*:

1. Tidak membutuhkan jumlah *cluster* di awal
2. Menghasilkan struktur hierarki
3. Cocok untuk eksplorasi data
4. Sensitif terhadap noise dan outlier

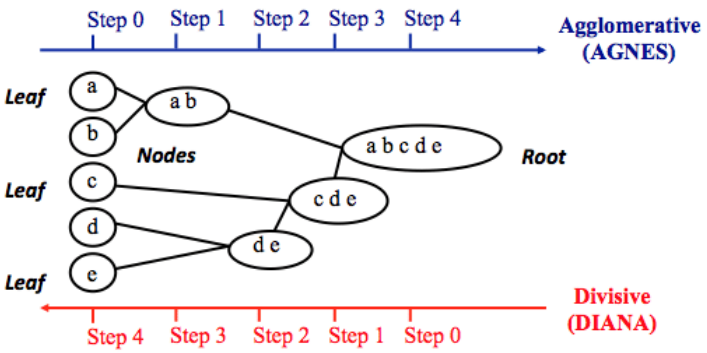
Tujuan *Hierarchical Clustering*:

1. Mengelompokkan data yang memiliki kemiripan tinggi
2. Menemukan struktur alami dalam data
3. Mendukung analisis eksploratif

B. Jenis *Hierarchical Clustering*

Gambar 1. Jenis *Hierarchical Clustering*

Ilustrasi:



Gambar 2. Ilustrasi Perbedaan tahapan antara *Agglomerative* dan *Divisive Clustering*

C. Metode/ Algoritma pada *Hierarchical Clustering*

Terdapat beberapa metode yang digunakan untuk membentuk *cluster* pada *hierarchical clustering*, yaitu:

1. Single Linkage (Nearest Neighbor)

Single linkage adalah metode hierarchical clustering yang mengukur kedekatan antar cluster berdasarkan jarak paling dekat antar anggotanya. Artinya, penggabungan dilakukan jika ada satu pasangan titik yang jaraknya paling kecil. Prosesnya dimulai dari setiap data sebagai cluster, lalu digabung secara bertahap berdasarkan jarak terdekat. Ciri utama dari metode ini adalah munculnya *chaining effect*, yaitu cluster cenderung memanjang karena penggabungan tetap terjadi selama masih ada titik yang berdekatan. Metode ini sederhana dan bisa menangkap pola tidak beraturan, tetapi sensitif terhadap noise sehingga hasilnya kadang kurang kompak.

2. Complete Linkage (Farthest Neighbor)

Complete linkage merupakan metode hierarchical clustering yang menilai jarak antar cluster dari titik yang paling berjauhan di dalamnya. Dengan kata lain, penggabungan cluster mempertimbangkan kondisi terburuk (jarak maksimum) antar anggota. Prosesnya dimulai dengan setiap data sebagai cluster tersendiri, kemudian dua cluster akan digabung jika jarak maksimumnya paling kecil dibandingkan pasangan lain. Langkah ini dilakukan secara bertahap. Metode ini cenderung menghasilkan cluster yang padat dan terstruktur. Namun, sifatnya yang ketat membuat penggabungan bisa tertunda hanya karena adanya satu titik yang letaknya jauh.

3. Average Linkage

Average linkage adalah metode hierarchical clustering yang menentukan jarak antar cluster dari nilai rata-rata seluruh pasangan titik di dalamnya, sehingga tidak hanya bergantung pada satu titik tertentu. Pengelompokan dilakukan secara bertahap dengan menggabungkan dua cluster yang memiliki rata-rata jarak paling kecil. Proses ini terus berlangsung hingga terbentuk struktur cluster yang diinginkan. Pendekatan ini menghasilkan cluster yang relatif seimbang dan tidak terlalu ekstrem. Namun, karena menggunakan rata-rata, metode ini membutuhkan perhitungan lebih banyak dan kadang kurang peka terhadap pola tertentu dalam data.

4. Ward

Ward's method merupakan pendekatan dalam hierarchical clustering yang menekankan pada pembentukan cluster dengan tingkat kemiripan tinggi melalui pengendalian varians di dalamnya. Penggabungan dilakukan secara bertahap dengan memilih pasangan cluster yang memberikan perubahan varians paling kecil. Hasilnya, cluster yang terbentuk cenderung lebih rapi, kompak, dan anggotanya relatif seragam. Namun, metode ini cukup peka terhadap outlier dan biasanya lebih sesuai digunakan dengan jarak Euclidean.

Tabel 1. Perbedaan antar Metode pada *Hierarchical Clustering*

Metode	Dasar Perhitungan	Karakteristik Utama
Single Linkage	Jarak terdekat $d(x,y) = \min_{i \in x, j \in y} (d_{i,j})$	Cenderung membentuk rantai (chaining), sederhana, bisa mendeteksi pola sederhana, sensitif terhadap *noise* dan *outlier*
Complete Linkage	Jarak terjauh $d(x,y) = \max_{i \in x, j \in y} (d_{i,j})$	Hasil cluster cenderung rapat, bentuk lebih rapi dari pada single linkage, sensitif terhadap titik/ objek yang jauh (*outlier* maupun tidak)
Average Linkage	Rata-rata jarak $d(x,y) = \frac{1}{\lvert x \rvert \lvert y \rvert} \sum_{i \in x} \sum_{j \in y} d_{i,j}$	Lebih seimbang dan stabil, perhitungan lebih kompleks, hasil yang diperoleh cukuk konsisten dan tidak ekstrim
Ward's method	Minimasi varians $d(x,y) = \Delta SSE$	Hasil cluster cenderung homogen & paling rapi dari 3 metode yang lain, sensitif terhadap *outlier*, hanya bisa menggunakan perhitungan jarak Euclidean.

D. Tahapan dalam *Hierarchical Clustering*

Berikut adalah tahapan/ proses kinerja dari *hierarchical clustering*:

1. Data Preparation
   - Data yang digunakan harus numeric
   - Perlu adanya encoding untuk data kategorikal
   - Perlu ada standarisasi (secara umum standarisasi dengan normal baku)

     Jika varians populasi diketahui:

     $$Z = \frac{X - \mu}{\sigma}$$

     Jika varians populasi tidak diketahui:

     $$Z = \frac{X - \bar{X}}{S}$$

2. Menghitung jarak antar objek

   Terdapat tiga jenis jarak yang sering digunakan, yaitu:
   - Euclidean, rumus secara general

$$d(x,y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

- Manhattan, rumus secara general

$$d(x,y) = \sum_{i=1}^{n}|x_i - y_i|$$

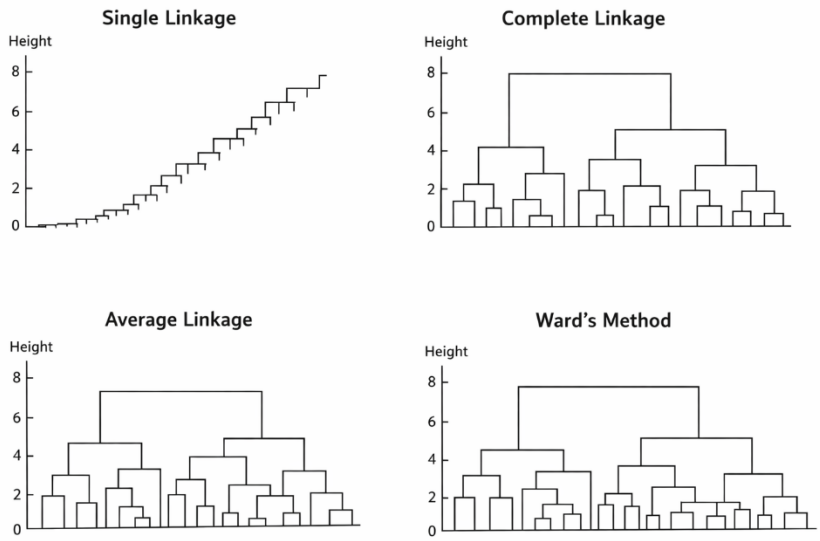
- Cosine, rumus secara general

$$\cos(\theta) = \frac{x \cdot y}{\|x\|\|y\|}$$

Keterangan:
x, y : variabel yang digunakan dalam *clustering*
n : jumlah total record/ objek

3. Menentukan likage
   Perlu memilih salah satu antara empat metode yang ada, yaitu single linkage, complete linkage, average linkage, atau ward's method
4. Melakukan proses cluster
   - Menggabungkan dua cluster dengan jarak yang bersesuaian
   - Memperbarui matriks jarak
   - Mengulangi sampai data tergabung dalam cluster
5. Visualisasi dendogram
   - Menunjukkan proses penggabungan cluster
   - Digunakan untuk menentukan jumlah cluster

   Contoh dendogram:



Gambar 3. Bentuk Dendogram dari Empat Metode *Hierarchical Clustering*

6. Menentukan jumlah cluster yang disesuaikan dengan kebutuhan

Partitioning Clustering

A. Pendahuluan

Sama halnya dengan *hierarchical clustering, partitional clustering* juga mengelompokkan data ke dalam sejumlah cluster tertentu berdasarkan tingkat kemiripan (*similarity*) atau perbedaan (*dissimilarity*) antar objek. Bedanya, metode ini membagi data secara langsung ke dalam beberapa kelompok yang tidak saling tumpang tindih, di mana setiap objek hanya menjadi anggota dari satu cluster. Proses pengelompokan dilakukan dengan mengoptimalkan fungsi tertentu, seperti meminimalkan jarak antar data dalam satu *cluster* (*intra-cluster*) dan memaksimalkan jarak antar cluster (*inter-cluster*), sehingga diperoleh kelompok data yang homogen dan terpisah dengan jelas. Satu hal lagi, *partitional* atau *non hierarchical clustering* menentukan jumlah *cluster* di awal proses.

Karakteristik *Partitional Clusteting*:

1. Jumlah cluster (k) harus ditentukan di awal
2. Setiap data hanya berada pada satu *cluster* (*hard clustering*)
3. Menggunakan pusat *cluster* (*centroid* atau *medoid*)
4. Cocok untuk dataset besar karena relatif efisien

Tujuan *Partitional Clustering*:

1. Membagi data menjadi beberapa kelompok homogen
2. Memaksimalkan kesamaan dalam *cluster* (*intra-cluster*)
3. Meminimalkan perbedaan antar *cluster* (*inter-cluster*)

B. Metode *Partitional Clustering*

Metode atau algoritma pada *partitional clustering* yang sering digunakan untuk membentuk *cluster* adalah K-means *clustering* dan K-medoid *clustering*. Uraian dari masing-masing algoritma adalah:

1. K-means *Clustering*

K-means berfungsi untuk mengelompokkan/ membagi data menjadi ***k cluster*** berdasarkan kemiriripan/ kesamaan karakteristik dari beberapa variabel yang digunakan. Proses membentuk *cluster* pada K-means menggunakan ***centroid*** (titik pusat *cluster*) sebagai representasi dari masing-masing *cluster* yang terbentuk. Algoritma K-means ini bekerja dengan cara:

- Mengelompokkan data ke *centroid* terdekat
- Memperbarui *centroid* secara iterative
- Proses berhenti ketika mencapai kondisi konvergen, di mana hasil mendekati nilai tertentu (stabil) seiring bertambahnya iterasi, sehingga perubahan hasil antar-iterasi menjadi sangat kecil atau tidak signifikan.

Karakteristik K-means *Clustering*:

- *Hard cluster* (1 data = 1 *cluster*)
- Menggunakan centrodi (mean data dalam setiap *cluster*)

$$\mu_j = \frac{1}{n_j}\sum_{i=1}^{n_j} x_i$$

Keterangan:

μ_j : centroid pada cluster ke-j
n_j : banyaknya record/ objek pada tiap cluster
x_i : datar/ record/ objek ke-i pada variabel X

- Cepat dan efisien
- *Euclidean distance* paling cocok untuk menentukan jarak antar objek
- Sensitif terhadap inisialisasi awal
- Meminimalkan SSE dalam *cluster* atau *within-cluster sum of square error* (WCSS)

$$WCSS = \sum_{j=1}^{k}\sum_{x_i \epsilon C_j} \|x_i - \mu_j\|^2$$

Keterangan:
k : banyak *cluster*
C_j : *cluster* ke-j
μ_j : *centroid* dari *cluster* ke-j

2. K-medoid *Clustering*

K-Medoids adalah proses membentuk kelompok atau membagi data ke dalam k cluster yang ditentukan oleh *medoid*, yaitu data aktual yang paling representatif dalam suatu cluster. Berbeda dengan K-Means yang menggunakan rata-rata (*centroid*), K-Medoids menggunakan data asli sebagai pusat *cluster*, sehingga lebih tahan terhadap *outlier*. K-medoids ini cara kerjanya adalah:

- Mengelompokkan data ke *medoid* terdekat berdasarkan ukuran jarak
- Memperbarui medoid dengan memilih data actual yang dapat meminimalkan total jarak dalam cluster secara iteratif
- Proses berhenti ketika mencapai kondisi konvergen, yaitu ketika tidak terjadi perubahan medoid atau perubahan total jarak antar-iterasi sudah sangat kecil atau tidak signifikan.

Karakteristik K-medoids *Clustering*:

- *Hard clustering* (1 data = 1 *cluster*)
- Menggunakan medoid (memilih data actual yang paling representatif sebagai pusat *cluster*)
- Robust terhadap *outlier*, artinya tidak sensitif terhadap *outlier*. Sehingga cocok untuk data yang memiliki *outlier* tinggi
- Dapat menggunakan Euclidean atau Manhattan distance
- Meminimalkan total *dissimilarity*, yaitu meminimalkan total jarak antara data dan medoid:

$$J = \sum_{j=1}^{k}\sum_{x_i \epsilon C_j} d(x_i, m_j)$$

Keterangan:

m_j : *medoid cluster* ke-j

$d(x_i, m_j)$: jarak antar data dan medoid

Dissimilarity ini menjadi bagian evaluasi K-medoid *clustering* apakah hasilnya sudah optimal atau belum. Semakin kecil nilai J dari berbagai k *cluster* yang diujicoba, maka hasil *clustering* tersebut yang optimal.

Berdasarkan penjelasan di atas, maka berikut perbedaan antara K-means dan K-medoids *clustering*.

Tabel 2. Perbedaan antara K-means dan K-medoids *Clustering*

Aspek	K-means	K-medoids
Pusat *cluster*	Centroid (rata-rata data dari setiap *cluster*)	Medoid (data asli/ aktual dari setiap *cluster* yang representatif
Representasi *cluster*	Tidak selalu data aktual	Selalu data aktual
Fungsi objektif	Meminimalkan SSE (kuadrat jarak)	Meminimalkan total *dissimilarity*
Ketahanan terhadap *outlier*	Sensitif	Lebih robust
Kecepatan	Lebih cepat	Lebih lambat
Kompleksitas komputasi	Rendah	Tinggi
Inisialisasi	*Centroid* acak	*Medoid* data yang representatif
Jenis data	Disarankan untuk data numeric	Bisa numeric atau categorical
Skala data	Cocok untuk data besar	Cocok untuk data kecil menengah

C. Tahapan pada *Partitional Clustering*

Tabel 3. Tahapan *Partitional Clustering*

K-means	K-medoids
Data preparation (standarisasi wajib ada)	*Data preparation* (standarisasi wajib ada)
Menentukan jumlah *cluster* (k)	Menentukan jumlah *cluster* (k)
Inisialisasi awal berupa centroid acak (random dari dataset atau bukan dari dataset)	Inisialisasi awal memilih medoid dari dataset secara acak
Menghitung jarak data ke centroid	Menghitung jarak data ke medoid
Mengelompokkan berdasarkan data masuk ke cluster dengan centroid terdekat	Mengelompokkan berdasarkan data masuk ke cluster dengan medoid terdekat
Menghitung ulang centroid	Memilih data yang meminimalkan total jarak sebagai medoid baru
Mengecek perubahan centroid	Mengecek perubahan medoid
Iterasi berhenti ketika tidak ada perubahan centroid	Iterasi berhenti ketika tidak ada perubahan medoid

D. Evaluasi Hasil *Partitional Clustering*

Evaluasi hasil *clustering* merupakan tahap penting untuk menilai kualitas pengelompokan data yang dihasilkan oleh algoritma *partitional clustering* seperti K-Means dan K-Medoids. Hal ini ingin menunjukkan bahwa apakah jumlah *cluster* yang ditetapkan di awal itu sudah optimal atau belum. Karena *clustering* bersifat *unsupervised learning*, maka tidak tersedia label asli sebagai pembanding. Oleh karena itu, evaluasi dilakukan berdasarkan:

- Struktur internal data
- Tingkat kemiripan dalam cluster
- Tingkat perbedaan antar cluster

Tujuan Evaluasi Clustering

- Menilai apakah cluster yang terbentuk homogen (kompak)
- Memastikan cluster terpisah dengan baik (separated)
- Menentukan jumlah cluster yang optimal
- Membandingkan performa antar metode clustering

Jenis Evaluasi Clustering

1. ***Internal Evaluation***
   - *Sum of Squared Errors* (SSE), yaitu mengukur total jarak kuadrat antara data dan pusat *cluster* (*centroid*). Untuk formula/ rumus seperti pada WCSS. Kriteria hasil cluster optimal adalah nilai SSE yang semaikin kecil.
     Tahapannya adalah:
     - Menghitung SSE berbagai k, misalkan 2, 3, 4, 5, …, 10
     - Memilih SSE yang paling kecil

   - Silhouette Score, mengukur kualitas cluster berdasarkan cohesion dan separation. Rumus/ formulanya adalah:
     $$S_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$
     Keterangan:
     a_i : rata-rata jarak dalam cluster
     b_i : jarak ke cluster terdekat

     Interpretasi hasil S_i:
     $S_i \approx 1 \rightarrow$ hasil cluter sangat baik/ optimum
     $S_i \approx 0 \rightarrow$ hasil cluster overlap
     $S_i \approx 0 \rightarrow$ hasil cluster tidak sesuai/ salah cluster
     Tahapannya adalah:
     - Menghitung S_i berbagai k, misalkan 2, 3, 4, 5, …, 10
     - Memilih S_i yang paling mendekati 1

- Davies-Bouldin Index (DBI), mengukur rasio antara jarak dalam cluster dan antar cluster. Formula:

  $$DBI = \frac{1}{k}\sum_{i=1}^{k} \max_{j \neq i} \frac{S_i + S_j}{M_{ij}}$$

  Keterangan:
  M_{ij} : jarak antar centroid

  Interpretasi hasil DBI: hasil *cluster* optimum ketika DBI semakin kecil

  Tahapan:
  - Menghitung DBI dari berbagai k, misalkan 2, 3, 4, 5, …, 10
  - Memilih DBI yang paling kecil

2. *Relative Evaluation*

- Elbow method, untuk menentukan jumlah cluster optimal berdasarkan hasil titik perubahan jumlah cluster satu dengan yang lain yang membentuk siku. Tahapannya adalah:
  - Menghitung SSE berbagai k, misalkan 2, 3, 4, 5, …, 10
  - Membuat plot/ grafik antara SSE vs k
  - Memilih titik siku (elbow)/ titik perubahan signifikan menunjukkan clustering yang terbentuk sudah optimal

Ringkasan dari evaluasi untuk *partitioning clustering*.

Tabel 4. Evaluasi pada *Partitioning Clustering*

Metode	Tujuan	Cluster yang Optimal/ Optimum
SSE	Komparasi cluster	SSE semakin kecil
Silhouette	Cohesion & separation	S_i mendekati
DBI	Rasio cluster	DBI semakin kecil
Elbow	Penentuan k cluster	Titik siku
Dissimilarity (khusus K-medoid)	Total jarak	J semakin kecil

Contoh Syntax *Clustering*

A. Hierarchical Clustering

Sebagai contoh adalah *ward clustering*

```
import pandas as pd
import numpy as np
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
import matplotlib.pyplot as plt
import re

# ================================
# 1 Load Data
# ================================
data = pd.read_csv('Kesehatab.csv') -> nama file data

# ==============================================================
# 2 Standardisasi Data (Z-score ddof=1)-> jika menggunakan data sampel
# (jika menggunakan populasi, maka tidak perlu ada ddof atau ddof=0)
# ==============================================================
data_scaled = (data - data.mean()) / data.std(ddof=1)
data = pd.DataFrame(data_scaled, columns=data.columns)

# ================================
# 3 Membuat Label Variabel (optional)
# ================================
raw_labels = list(data.columns)
def extract_label(name):
    part_before_dash = name.split('-')[0].strip()
    match = re.search(r'\((.*?)\)', name)
    inside_parentheses = f"({match.group(1)})" if match else ''
    return f"{part_before_dash} {inside_parentheses}".strip()
labels = [extract_label(n) for n in raw_labels]

# ================================
# 4 Data untuk clustering (variabel)
# ================================
data_features = np.nan_to_num(data.values).T
n_samples = data_features.shape[0]

# ================================
# 5 Ward Hierarchical Clustering
# ================================
Z = linkage(data_features, method='ward')

# ================================
# 6 Plot Dendrogram
# ================================
plt.figure(figsize=(12,6))
dendro = dendrogram(
    Z,
    labels=labels,
    leaf_rotation=90
)
plt.title("Dendrogram Ward Cluster")
plt.xlabel("Variabel")
plt.ylabel("Distance")
plt.tight_layout()
plt.show()
leaf_order = dendro['leaves']

# ================================
# 7 Hitung SST
# ================================
mean_all = np.mean(data_features, axis=0)
SST = np.sum((data_features - mean_all)**2)
```

```
# ================================
# 8 Hitung R-Square
# ================================
r_squares = []
num_clusters_list = []
for k in range(n_samples, 0, -1):
    if k == n_samples:
        continue
    num_clusters_list.append(k)
    clusters = fcluster(Z, t=k, criterion='maxclust')
    SSE = 0
    for cluster_id in np.unique(clusters):
        cluster_points = data_features[clusters == cluster_id]
        cluster_mean = np.mean(cluster_points, axis=0)
        SSE += np.sum((cluster_points - cluster_mean)**2)
    R2 = 1 - (SSE / SST)
    r_squares.append(R2)

# ================================
# 9 Distance dan Frequency
# ================================
distance = Z[:,2]
frequency = Z[:,3].astype(int)

# ================================
# 10 Tabel Proses Cluster
# ================================
df_process = pd.DataFrame({
    "Cluster1": cluster1_names,
    "Cluster2": cluster2_names,
    "Cluster": combine_cluster,
    "Distance": distance,
    "Frequency": frequency,
    "R-Squared": r_squares,
    "Number of Cluster": num_clusters_list
})

# ================================
# 11 Cluster Semua Level
# ================================
cluster_levels = []
ordered_variables = [labels[i] for i in leaf_order]
for k in range(n_samples-1, 0, -1):
    clusters = fcluster(Z, t=k, criterion='maxclust')
    ordered_clusters = [clusters[i] for i in leaf_order]
    df_level = pd.DataFrame({
        "Variable": ordered_variables,
        f"Cluster_{k}": ordered_clusters
    })
    cluster_levels.append(df_level)
df_cluster_all = cluster_levels[0]
for df_temp in cluster_levels[1:]:
    df_cluster_all = df_cluster_all.merge(df_temp, on="Variable")

# ================================
# 14 Export Excel
# ================================
with pd.ExcelWriter("hasil_cluster_ward_final.xlsx") as writer:
    df_process.to_excel(writer, sheet_name="Proses_Cluster",
index=False)
    df_cluster_all.to_excel(writer, sheet_name="Cluster_All_Level",
index=False)
```

B. *Partitional Clustering*

Contohnya adalah K-means *clustering* dengan evaluasi cluster menggunakan Elbow

```
import pandas as pd
import numpy as np
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
import matplotlib.pyplot as plt
import re

# ================================
# 1 Load Data
# ================================
data = pd.read_csv('Kesehatab.csv') -> nama file data

# ==============================================================
# 2 Standardisasi Data (Z-score ddof=0)-> menggunakan data populasi
# (jika menggunakan sampel, maka ddof=1)
# ==============================================================
data_scaled = (data - data.mean()) / data.std(ddof=0)
data = pd.DataFrame(data_scaled, columns=data.columns)

# ================================
# 3 Membuat Label Variabel (optional)
# ================================
raw_labels = list(data.columns)
def extract_label(name):
    part_before_dash = name.split('-')[0].strip()
    match = re.search(r'\((.*?)\)', name)
    inside_parentheses = f"({match.group(1)})" if match else ''
    return f"{part_before_dash} {inside_parentheses}".strip()
labels = [extract_label(n) for n in raw_labels]

# ================================
# 4 Data untuk clustering (variabel)
# ================================
data_features = np.nan_to_num(data.values).T
n_samples = data_features.shape[0]

# ================================
# 5 Menentukan jumlah cluster (k)
# ================================
k = 3  # silakan ubah sesuai kebutuhan

# ================================
# 6 K-Means Clustering
# ================================
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
clusters = kmeans.fit_predict(data_features)
centroids = kmeans.cluster_centers_

# ================================
# 7 Hitung SSE (Inertia)
# ================================
SSE = kmeans.inertia_

# ================================
# 8 Hitung SST
# ================================
mean_all = np.mean(data_features, axis=0)
SST = np.sum((data_features - mean_all)**2)

# ================================
# 9 Hitung R-Square
# ================================
R2 = 1 - (SSE / SST)

# ================================
# 10 Tabel Hasil Cluster
# ================================
df_cluster = pd.DataFrame({
    "Variable": labels,
    "Cluster": clusters + 1
})

# ================================
# 11 Tabel Centroid
# ================================
df_centroid = pd.DataFrame(centroids)
df_centroid["Cluster"] = range(1, k+1)

# ================================
# 12 Elbow Method
# ================================
sse_list = []
K_range = range(1, n_samples + 1)
for k_val in K_range:
    km = KMeans(n_clusters=k_val, random_state=42, n_init=10)
    km.fit(data_features)
    sse_list.append(km.inertia_)
plt.figure(figsize=(8,5))
plt.plot(K_range, sse_list, marker='o')
plt.title("Elbow Method")
plt.xlabel("Jumlah Cluster (k)")
plt.ylabel("SSE")
plt.grid()
plt.show()

# ================================
# 13 Export Excel
# ================================
with pd.ExcelWriter("hasil_kmeans.xlsx") as writer:
    df_cluster.to_excel(writer, sheet_name="Cluster_Result",
index=False)
    df_centroid.to_excel(writer, sheet_name="Centroid", index=False)
    pd.DataFrame({
        "SSE": [SSE],
        "SST": [SST],
        "R-Square": [R2]
    }).to_excel(writer, sheet_name="Evaluation", index=False)
```

Referensi

Han, Jiaei, Jian Pei, and Hanghang Tong. 2023. *Data mining* Concepts and Techniques. 4th ed. Beth LoGiudice. Cambridge: Katey Birtcher.

Moreira, J. M., Carvalho, A. C. P. L. F., & Horváth, T. (2019). *A General Introduction to Data Analytics*. Hoboken, NJ: John Wiley & Sons, Inc.